

计算机系统基础

Lab0 实验报告

姓名： 吴向杰

学号： 25300190057

日期： 2026 年 9 月 21 日

目录

1	实验目的	1
2	实验环境	1
3	文档问题回答	1
4	实验过程	2
5	拓展阅读	6
6	为什么要学习 Git	10
7	实验总结	11

1 实验目的

本实验的主要目的是学习 Git 和 GitHub 的基本使用方法，理解版本控制的作用，并掌握仓库克隆、文件修改、暂存、提交、推送、分支创建、分支切换、分支合并以及合并冲突处理等基本操作。

通过本次实验，我还需要了解规范的提交信息和语义化版本等软件开发规范，初步认识 Git 在版本管理和多人协作中的作用。

2 实验环境

- 操作系统：Windows 与 WSL Ubuntu
- 代码编辑器：Visual Studio Code
- 版本控制工具：Git
- 远程仓库平台：GitHub
- GitHub 用户名：wuxiangjie84
- 实验仓库：<https://github.com/wuxiangjie84/ICS-Lab0>

3 文档问题回答

3.1 多人协同开发经历

我曾参加过数学建模竞赛，并在比赛过程中与队友共同完成模型分析、程序计算和论文撰写。虽然这不完全等同于正式的软件开发，但同样涉及多人分工、文件共享和成果整合。

当时我们主要通过微信传输代码、数据和论文文档。为了区分不同版本，通常只能在文件名中加入修改者姓名、日期或者“最新版”“最终版”等文字。这种协作方式比较直观，但当多人同时修改文件时，很容易出现版本错乱。例如，队员可能在不同旧版本上继续修改，或者无法确定某个文件是否已经包含其他人的最新成果。在整合论文和代码时，还需要人工比较多个文件，存在遗漏修改或覆盖他人成果的风险。

通过本次实验，我认识到 Git 能够为每次修改建立明确的提交记录，并保存修改时间、修改者和具体内容。不同成员还可以使用分支分别开展工作，完成后再通过合并功能整合成果。如果在数学建模竞赛中使用 Git，就能够更清楚地管理代码、数据处理脚本和论文源文件，减少通过微信反复传输文件导致的版本混乱，提高团队协作效率。

3.2 为什么 Git 要设计“暂存—提交”两个步骤

工作目录中可能同时存在多项修改，而这些修改不一定属于同一个任务。暂存区允许开发者从所有修改中选择一部分，将它们组织成一次内容明确的提交，而不是被迫把所有修改一次性保存。

例如，当代码文件和实验报告同时被修改时，可以先将代码加入暂存区并提交，再单独提交实验报告。这样每次提交都有清晰的主题，便于其他人检查，也便于以后查找问题或者回退某一部分修改。

因此，暂存和提交分为两个步骤，可以让开发者在真正保存版本之前进行筛选和整理，使提交历史更加清晰。

3.3 `git branch` 和 `git branch -a` 的区别

`git branch` 主要用于查看本地分支，其中带星号的分支表示当前所在分支。

`git branch -a` 中的 `-a` 表示 all，它不仅显示本地分支，还会显示远程跟踪分支，例如 `remotes/origin/main`。

因此，两者的主要区别是：

- `git branch`: 查看本地分支；
- `git branch -a`: 查看本地分支和远程跟踪分支。

4 实验过程

4.1 配置 Git 用户信息

首先在 WSL Ubuntu 中检查 Git 是否安装成功：

```
git --version
```

随后设置提交记录中使用的用户名和邮箱：

```
git config --global user.name "wuxiangjie84"  
git config --global user.email "wuxiangjie84@gmail.com"
```

这里的用户名和邮箱会被记录在之后创建的提交中。

4.2 配置 GitHub SSH 密钥

为了在本地安全地访问 GitHub，我使用以下命令生成 ED25519 类型的 SSH 密钥：

```
ssh-keygen -t ed25519 -C "wuxiangjie84@gmail.com"
```

密钥生成后，使用下面的命令显示公钥：

```
cat ~/.ssh/id_ed25519.pub
```

我将公钥添加到 GitHub 的 SSH and GPG keys 设置页面，并使用以下命令测试连接：

```
ssh -T git@github.com
```

GitHub 成功识别账号后，说明 SSH 配置完成。

4.3 创建并克隆个人仓库

我打开课程提供的模板仓库，依次点击 Use this template，然后选择 Create a new repository，创建个人仓库 ICS-Lab0。

随后在 WSL 中执行：

```
git clone git@github.com:wuxiangjie84/ICS-Lab0.git
cd ICS-Lab0
code .
```

这样便在 Visual Studio Code 的 WSL 模式中打开了本地仓库。

4.4 完成 TODO 并进行第一次提交

模板中的 main.c 原本包含以下代码：

```
printf("Hello, world!\n");
```

我将它修改为：

```
printf("Hello, this is my first ICS lab!\n");
```

保存文件后执行：

```
git status
git diff
git add main.c
```

```
git commit -m "Complete TODO in main.c"
git push origin main
```

其中, `git add` 将修改加入暂存区, `git commit` 创建本地提交, `git push` 将本地提交上传到 GitHub。

4.5 创建 feature 分支

我从 `main` 分支创建并切换到 `feature` 分支:

```
git switch -c feature
```

随后将输出语句修改为:

```
printf("This message comes from feature branch.\n");
```

保存并提交:

```
git add main.c
git commit -m "Change message on feature branch"
```

4.6 在 `main` 分支进行另一项修改

完成 `feature` 分支的提交后, 我切换回 `main` 分支:

```
git switch main
```

然后修改 `main.c` 中相同的代码行:

```
printf("This message comes from main branch.\n");
```

保存并提交:

```
git add main.c
git commit -m "Change message on main branch"
```

此时, `main` 和 `feature` 分支都修改了 `main.c` 的同一位置, 但修改结果不同。

4.7 合并分支并产生冲突

在 `main` 分支执行:

```
git merge feature
```

Git 输出了类似下面的信息：

```
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

产生冲突的原因是两个分支分别修改了同一个文件中的同一行，Git 无法自动判断应该使用哪个分支的内容，因此需要由开发者手动处理。

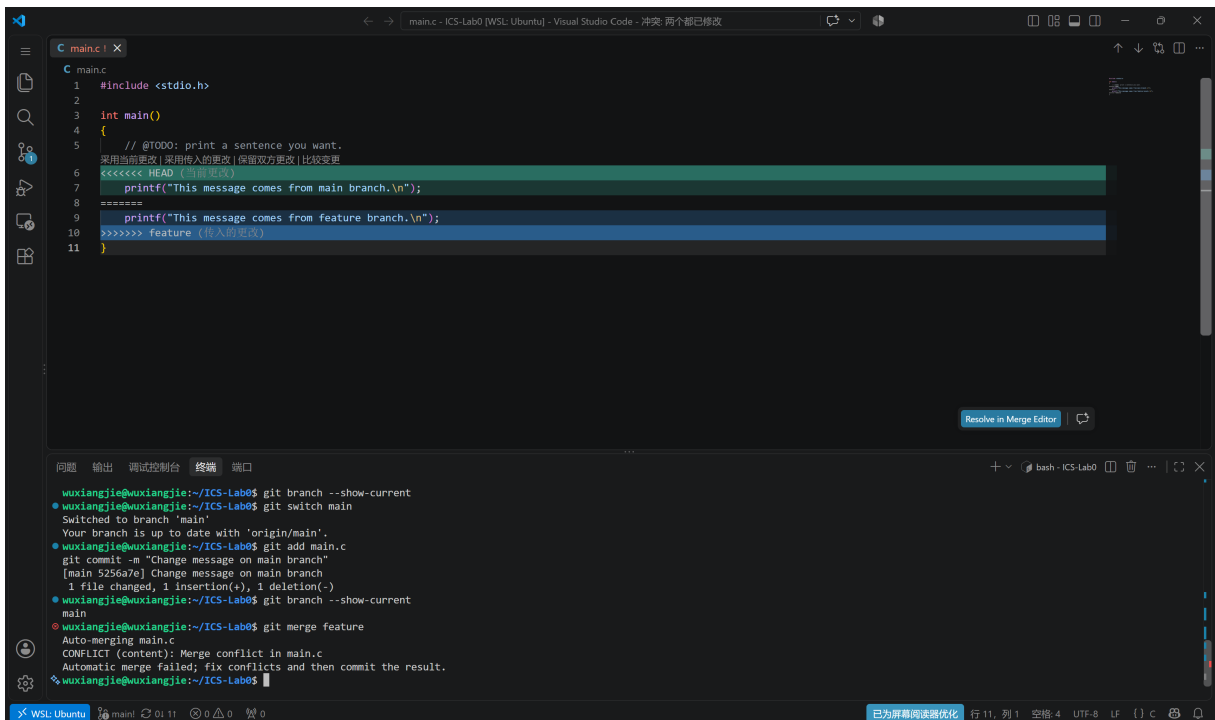


图 1: 合并时产生的冲突

4.8 解决合并冲突

冲突发生后，文件中出现了以下标记：

```
<<<<<<< HEAD
main branch content
=====
feature branch content
>>>>>>> feature
```

其中，HEAD 部分表示当前 main 分支的内容，下半部分表示准备合并的 feature 分支内容。

我在 Visual Studio Code 中选择保留双方更改，最终保留两条输出语句：

```
printf("This message comes from main branch.\n");
printf("This message comes from feature branch.\n");
```

确认冲突标记已经完全删除并保存文件后，执行：

```
git add main.c
git commit -m "Resolve merge conflict between main and feature"
git push origin main
```

最后使用以下命令查看分支和提交历史：

```
git log --graph --oneline --all
```

提交图显示 main 和 feature 分支分别产生了提交，并最终汇合到一次合并提交中。



```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
* 057f267 (HEAD -> main, origin/main, origin/HEAD) Save resolved main.c
* 3a8d302 Resolve merge conflict between main and feature
| \
| * 3aecce2 (feature) Change message on feature branch
* | 5256a7e Change message on main branch
| /
* fabaa06 Complete TODO in main.c
* ff42ebd Initial commit
wuxiangjie@wuxiangjie:~/ICS-Lab0$
```

图 2: 解决冲突后的提交历史

5 拓展阅读

5.1 Commit Message 规范

参考资料：阮一峰：《Commit message 和 Change log 编写指南》

5.1.1 文章内容概括

Commit message 是创建 Git 提交时填写的说明，用于解释本次提交完成了什么修改。虽然 Git 对提交信息的具体内容没有严格限制，但如果开发者只填写 update、

change 或者 fix 等含义模糊的词，其他人以后查看提交历史时，就很难理解每次修改的真正目的。

规范的 Commit message 能够提供清晰的项目历史。开发者可以通过提交信息快速了解某次修改属于新增功能、错误修复、文档更新还是代码重构，也可以按照提交类型筛选历史记录。对于规模较大的项目，规范的提交信息还可以被工具读取，并自动生成 Change Log。

文章介绍的提交格式由 Header、Body 和 Footer 三部分组成。其中 Header 是必需的，通常采用以下格式：

```
<type>(<scope>): <subject>
```

type 表示修改类型，常见类型包括：

- feat: 增加新功能；
- fix: 修复程序错误；
- docs: 修改说明文档；
- style: 修改代码格式，但不改变程序行为；
- refactor: 进行代码重构；
- test: 增加或修改测试；
- chore: 修改构建过程或辅助工具。

scope 用于说明本次修改影响的范围，可以省略；subject 则用一句简短的话说明修改目的。如果简单的一行无法解释清楚，还可以在 Body 中补充修改原因、实现方式以及修改前后的差别。Footer 通常用于说明不兼容修改，或者关联、关闭某个 Issue。

5.1.2 结合本次实验的理解

在本次实验中，我使用过以下提交信息：

```
Complete TODO in main.c  
Change message on feature branch  
Change message on main branch  
Resolve merge conflict between main and feature
```

这些提交信息基本能够说明每次操作的目的，因此查看 git log 时，可以根据文字判断不同提交分别完成了什么任务。但是，按照文章介绍的规范，还可以进一步写成：

```
feat(main): complete output message
feat(feature): change output on feature branch
fix(merge): resolve conflict in main.c
docs(report): add Lab0 experiment report
```

这种格式在本次只有几个提交的小实验中差别并不明显，但当个项目存在几百甚至几千次提交时，统一格式能够显著降低理解成本。例如，看到 `docs` 就可以知道修改只涉及文档，看到 `fix` 就可以知道该提交修复了错误，不必逐个打开文件查看具体内容。

我认为 Commit message 的重要性不只在于“给这次提交起一个名字”，而在于向未来的自己和其他协作者解释修改意图。代码只能反映“程序现在是什么样”，而好的提交信息能够进一步说明“为什么要这样修改”。当程序出现问题时，开发者可以根据提交信息快速缩小排查范围，也可以更容易判断某次提交是否适合回退或迁移到其他分支。

因此，提交信息实际上是代码历史的一部分。一个内容正确但提交记录混乱的项目，仍然会给维护和协作带来困难。养成编写清晰 Commit message 的习惯，能够使版本历史真正发挥作用。

5.2 语义化版本

参考资料：Semantic Versioning 2.0.0 中文版

5.2.1 文章内容概括

语义化版本使用 `x.y.z` 的形式表示软件版本，其中 `x`、`y` 和 `z` 分别表示主版本号、次版本号和修订号。

版本号的基本递增规则如下：

- 当软件进行了不向下兼容的公共 API 修改时，增加主版本号；
- 当软件增加了向下兼容的新功能时，增加次版本号；
- 当软件只进行了向下兼容的问题修复时，增加修订号。

例如，软件从 `1.2.3` 升级到 `1.2.4`，通常表示修复了一些问题，但原有使用方式仍然有效；升级到 `1.3.0`，通常表示增加了新功能，同时仍然兼容原有功能；升级到 `2.0.0`，则意味着可能存在与旧版本不兼容的重要改变。

语义化版本还允许使用先行版本标识，例如：

```
1.0.0-alpha  
1.0.0-beta.1  
1.0.0-rc.1
```

这些标识表示软件尚未达到对应正式版本的稳定程度。版本号后面还可以增加构建信息，但构建信息通常不影响版本之间的优先级。

语义化版本要求已经发布的某个版本不能被直接修改。如果发布后发现问题，应当通过发布一个新的版本进行修正，而不是在保持版本号不变的情况下替换原来的内容。这样才能保证同一个版本号始终对应同一份确定的软件。

5.2.2 结合实际开发的理解

在阅读之前，我更容易把版本号理解为软件更新次数或者开发者自行决定的编号。阅读之后，我认识到语义化版本的核心不是数字本身，而是利用版本号向使用者传达兼容性信息。

假设一个软件依赖某个函数库。如果函数库只是从 1.2.3 升级到 1.2.4，使用者通常可以判断这只是兼容的问题修复，升级风险相对较低。但如果版本从 1.2.3 升级到 2.0.0，使用者就应当注意公共接口可能发生了不兼容变化，原有程序可能需要修改后才能继续使用。

因此，语义化版本实际上可以被看作开发者与使用者之间的一种约定。版本号不是为了表示“新版本比旧版本更高级”，而是为了说明修改对已有使用方式产生了多大影响。如果开发者没有按照统一规则更新版本号，使用者就无法从版本号判断升级风险，自动化依赖管理也会失去可靠依据。

如果将语义化版本应用到一个简单的学习项目中，可以进行如下设计：

- 0.1.0：项目处于初始开发阶段；
- 0.2.0：增加了一项新功能；
- 0.2.1：修复了该功能中的小错误；
- 1.0.0：项目功能稳定，形成第一个正式版本；
- 2.0.0：对外部使用方式进行了不兼容的重大修改。

虽然本次 Lab0 的程序非常简单，并不需要真正发布多个版本，但语义化版本所体现的思想仍然适用：开发者不仅要记录“发生了修改”，还应当说明修改的性质和影响范围。

5.3 两篇材料之间的联系与我的理解

Commit Message 规范和语义化版本关注的是版本管理中的不同层次。

Commit message 主要描述一次具体提交，回答的是“这一次修改做了什么、为什么要修改”。一个项目在一天之内可能产生多次提交，每次提交都应当有清晰的说明。

语义化版本主要描述一次软件发布，回答的是“这批修改对软件使用者有什么影响”。一个正式版本通常包含多个提交，只有当这些修改被整理和验证之后，才会形成新的软件版本。

因此，可以将二者理解为：

- Commit message 管理项目内部的细粒度修改历史；
- 语义化版本向使用者表达一次发布的整体影响；
- Git 保存实际的提交和分支关系；
- GitHub 等平台负责远程存储、共享和协作。

以本次实验为例，我先在 `feature` 和 `main` 分支中分别创建提交，再将两个分支合并。每个 Commit message 用来解释对应步骤，而如果将合并后的程序作为一个正式成果发布，还可以为它设置一个版本号。

通过阅读这两篇材料，我认识到版本控制不只是把文件上传到 GitHub。Git 负责保存修改历史，但历史是否容易理解，还取决于开发者是否编写了清晰的提交信息；软件是否能够被安全升级，也取决于开发者是否使用了有明确含义的版本号。

规范的目的是并不是增加形式上的负担，而是减少未来沟通、排错和维护的成本。在个人小项目中，这种收益可能暂时不明显；但在数学建模竞赛或者多人项目中，如果代码、论文和数据处理脚本需要频繁修改，清晰的提交记录和版本约定就能有效减少文件错乱、重复修改和错误覆盖等问题。

6 为什么要学习 Git

Git 首先能够完整记录文件的修改历史。与手动保存“最终版”“最终版 2”“真正最终版”等多个文件相比，Git 可以通过提交记录明确表示每个版本的修改时间、修改内容和作者，并在出现问题时恢复到之前的版本。

其次，Git 的分支功能允许开发者在不影响主分支的情况下尝试新的修改。本次实验中，`feature` 分支和 `main` 分支可以分别保存不同内容，说明不同开发任务可以在相对独立的环境中进行。

再次，Git 能够支持多人协作。当多人修改不同部分时，Git 可以自动合并；当多人修改同一位置时，Git 会明确指出冲突，并要求开发者确认最终结果，从而避免不知情地覆盖他人的工作。

GitHub 等远程仓库平台还提供了代码备份和共享能力。本地代码即使出现问题，也可以从远程仓库重新获得。因此，学习 Git 不只是学习几个命令，更是在学习如何有组织地管理代码、控制修改风险以及与其他开发者协作。

7 实验总结

本次实验中，我完成了 Git 用户信息和 GitHub SSH 密钥的配置，使用课程模板创建并克隆了个人仓库，修改了 `main.c`，并完成了文件暂存、提交和推送操作。

在分支实验中，我分别在 `main` 和 `feature` 分支修改了同一行代码，从而主动产生合并冲突。随后，我观察了 Git 的冲突标记，手动选择需要保留的内容，删除冲突标记并完成合并提交。

通过本次实验，我对工作目录、暂存区、本地提交、远程仓库以及分支之间的关系有了初步理解，也认识到清晰的提交记录和版本规范对于长期维护项目的重要性。